`Cache-Control` is the primary HTTP response/request header used to control **whether a response can be cached, where it may be cached, and when the cached response can be reused**.

For an architect, the most useful way to think about it is:

> **Cache-Control defines the caching contract between the origin server, browsers, CDNs, and other HTTP caches.**

It is especially important because caching is not simply "cache this response for N seconds." Modern HTTP caching lets you control **freshness, revalidation, shared vs. private caches, authorization-sensitive responses, and stale behavior**.

---

# 1. The basic model

Suppose your API returns:

```
HTTP/1.1 200 OK
Cache-Control: max-age=300
Content-Type: application/json

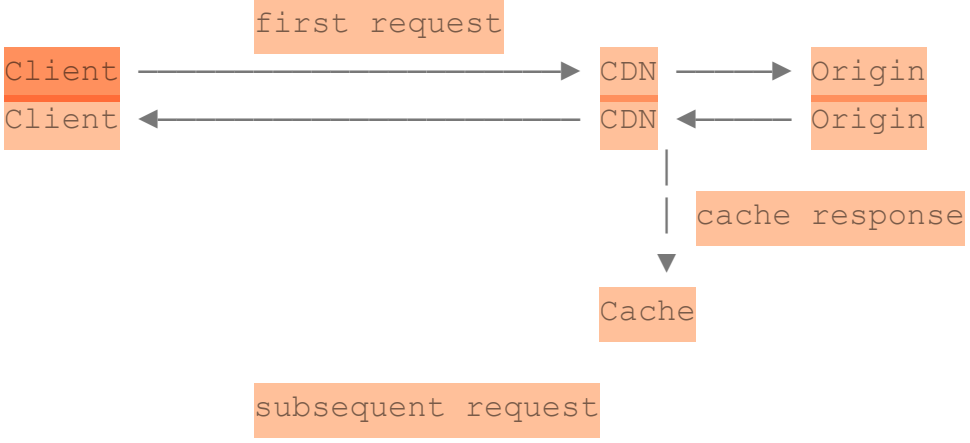
{ ... }
```

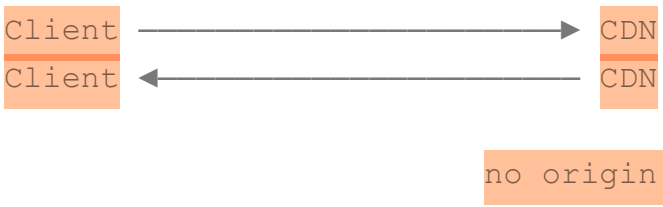
`max-age=300` means:

> A cache may consider this response **fresh for 300 seconds** after it was generated.

During those 5 minutes, a browser/CDN can potentially serve the cached response without contacting your origin.

Conceptually:






This can dramatically reduce:

- latency
- origin traffic
- database load
- compute cost
- bandwidth

---

# 2. The most important directives

The directives you'll use most often are:

| Directive | Meaning |
|---|---|
| `max-age=N` | Response is fresh for N seconds |
| `s-maxage=N` | Freshness lifetime for shared caches/CDNs |
| `public` | Response may be stored by shared caches |
| `private` | Response should only be stored by a private cache such as the browser |
| `no-cache` | Cache may store it, but must revalidate before reuse |
| `no-store` | Do not store the response |
| `must-revalidate` | Once stale, don't reuse without successful revalidation |
| `immutable` | Response won't change during its freshness lifetime |
| `stale-while-revalidate=N` | Serve stale response while asynchronously revalidating |
| `stale-if-error=N` | Permit stale response when origin fails |
| `no-transform` | Don't transform the representation |

One of the most important distinctions is:

### `no-cache` ≠ `no-store`

This causes a lot of production mistakes.

### `no-cache`

```
Cache-Control: no-cache
```

means:

> You **may store** this response, but you must validate it with the origin before using it.

It does **not** mean "don't cache."

### `no-store`

```
Cache-Control: no-store
```

means:

> Don't store this response.

This is appropriate for genuinely sensitive or non-cacheable responses.

---

## 3. Static assets

Static assets are usually the easiest things to cache.

For example:

```
/app.js
/styles.css
/logo.svg
/font.woff2
```

If the filenames are content-hashed:

```
app.a83f91c2.js
styles.91af32d1.css
```

you can use extremely aggressive caching:

```
Cache-Control: public, max-age=31536000, immutable
```

That's approximately one year.

Why is this safe?

Because when the content changes, you don't overwrite:

```
app.a83f91c2.js
```

You generate:

```
app.b72c91f4.js
```

So:

```
Old URL → old content
New URL → new content
```

This is one of the most powerful caching patterns for web applications.

## Recommended pattern

```
HTML
 │
 ├── app.a83f91c2.js
 ├── styles.91af32d1.css
 └── logo.72a1f4.svg
```

The assets:

```
Cache-Control: public, max-age=31536000, immutable
```

The HTML can have a much shorter cache lifetime.

# 4. Frequently changing API data

Suppose:

```
GET /api/products
```

changes frequently.

You might use:

```
Cache-Control: public, max-age=60
```

Meaning:

> A cache can serve this response for 60 seconds.

This is useful when a small amount of staleness is acceptable.

For example:

```
Product catalog
├── origin update
├── CDN cache: 60 sec
└── users may see up to ~60 sec old data
```

This can be an excellent trade-off.

---

# 5. Public API data with different browser/CDN requirements

Suppose you want:

- browser cache: 30 seconds
- CDN cache: 5 minutes

Use:

```
Cache-Control: public, max-age=30, s-maxage=300
```

The distinction is important:

- `max-age` → generally applies to caches, including browser caches
- `s-maxage` → overrides freshness for **shared caches**

So:

```
Browser       → 30 sec
CDN           → 300 sec
```

This is extremely useful for API architectures using CDNs.

---

# 6. User-specific responses

Suppose:

```
GET /api/me
```

returns:

```
{
  "name": "Alice",
  "email": "alice@example.com"
}
```

You generally don't want a shared CDN cache serving Alice's response to Bob.

Use:

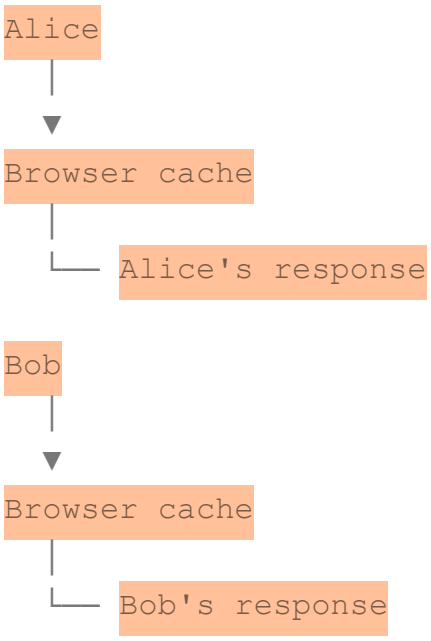
```
Cache-Control: private, max-age=60
```

This tells shared caches:

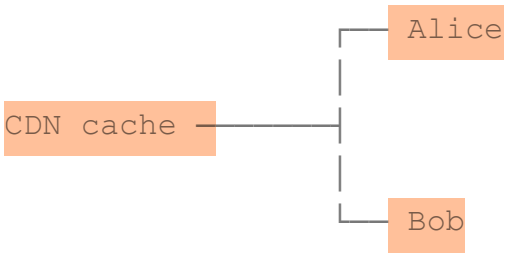
> Don't store this response as a shared response.

The browser may still cache it for 60 seconds.

Architecture:




rather than:




---

# 7. Highly sensitive responses

For things such as:

- payment information
- authentication responses
- highly sensitive personal data
- security-sensitive operations

you may want:

```
Cache-Control: no-store
```

This tells caches not to store the response.

For example:

```
HTTP/1.1 200 OK
Cache-Control: no-store
Content-Type: application/json
```

Be careful, however, not to reflexively use `no-store` everywhere. It eliminates useful caching opportunities.

---

# 8. `no-cache`: when you want validation

Suppose you have a resource that can be cached but should always be checked for freshness.

```
Cache-Control: no-cache
```

The cache can retain the response but must revalidate before reusing it.

This works particularly well with validators such as:

```
ETag: "abc123"
```

The next request can become:

```
GET /document
If-None-Match: "abc123"
```

If nothing changed:

```
HTTP/1.1 304 Not Modified
```

The server doesn't have to send the entire response again.

This gives you:

```
Cache storage
      +
cheap validation
      =
efficient freshness
```

## 9. ETag + no-cache

This is an important architectural pattern.

Imagine:

```
Cache-Control: no-cache
ETag: "v42"
```

The browser stores the response.

Next time:

```
GET /api/config
If-None-Match: "v42"
```

Server checks:

```
Current version = v42
Requested version = v42
```

and responds:

```
304 Not Modified
```

Instead of:

```
200 OK
10 KB JSON
```

you get:

```
304 Not Modified
```

This reduces bandwidth while preserving strong freshness semantics.

---

# 10. Last-Modified

Another validation mechanism is:

```
Last-Modified: Wed, 26 Aug 2026 08:00:00 GMT
```

The client can subsequently send:

```
If-Modified-Since: Wed, 26 Aug 2026 08:00:00 GMT
```

and the server can return:

```
304 Not Modified
```

In modern systems, `ETag` is often preferable because it provides a more precise representation validator.

---

# 11. stale-while-revalidate

This is one of my favorite directives for high-scale systems.

Suppose:

```
Cache-Control: public, max-age=60, stale-while-revalidate=300
```

Interpretation:

```
0–60 sec
    Fresh → serve immediately

60–360 sec
    Stale but usable → serve immediately
    + revalidate asynchronously

>360 sec
    Too stale → normal revalidation required
```

This gives you a **stale-while-revalidate** pattern.

Imagine a CDN receives a request at T=120 seconds.

Instead of:

```
Client
  ↓
CDN
  ↓
Origin
  ↓
wait...
```

it can:

```
Client ──► CDN ──► stale response immediately
                │
                └──► Origin revalidation
```

This is excellent for:

- news feeds
- product catalogs
- dashboards
- public content
- configuration
- recommendation data

where slight staleness is acceptable but low latency is valuable.

---

# 12. `stale-if-error`

Another powerful resilience mechanism:

```
Cache-Control: public, max-age=60, stale-if-error=600
```

Means:

> If the cached response becomes stale but the origin is failing, the cache may serve the stale response for up to another 10 minutes.

So:

```
Origin healthy
      |
      ▼
fresh response

Origin unavailable
      |
      ▼
serve stale cached response
```

This can turn caching into a **resilience mechanism**, not merely a performance optimization.

For example:

```
Cache-Control: public, max-age=60, stale-if-error=600
```

can allow a public API to continue functioning during a short origin outage.

---

# 13. `must-revalidate`

Consider:

```
Cache-Control: max-age=60, must-revalidate
```

After 60 seconds, the response is stale.

The cache cannot simply decide:

> "I'll return the stale value anyway."

It must successfully revalidate it before reuse.

This is useful when stale data is not acceptable.

---

# 14. `public` vs `private`

This distinction becomes especially important with CDNs.

## Public

```
Cache-Control: public, max-age=300
```

Suitable for:

```
GET /products
GET /articles/123
GET /images/logo.png
GET /api/countries
```

## Private

```
Cache-Control: private, max-age=300
```

Suitable for:

```
GET /profile
GET /my-orders
GET /dashboard
```

The browser can cache it, but a shared cache shouldn't.

---

# 15. Authenticated APIs require particular care

Suppose:

```
Authorization: Bearer eyJ...
```

is sent.

You should not blindly assume that adding `Authorization` automatically produces the caching semantics you want across every intermediary.

For user-specific API responses, a conservative choice is:

```
Cache-Control: private, no-cache
```

or, if nothing should be stored:

```
Cache-Control: no-store
```

If you deliberately want shared caching of authenticated/publicly equivalent responses, you need a much more carefully designed cache key and authorization model.

This is an important architectural boundary:

> **Never allow user identity to accidentally fall outside the cache key.**

Otherwise you risk the classic catastrophic failure:

```
User A → CDN → cached response for A

User B → CDN → receives A's response
```

---

# 16. Cache keys matter as much as Cache-Control

Consider:

```
GET /products?category=books
```

and:

```
GET /products?category=electronics
```

A cache generally needs to distinguish those requests.

Similarly:

```
Accept-Language: en
Accept-Language: fr
```

may require different representations.

That's where:

```
Vary: Accept-Language
```

comes into play.

For example:

```
Cache-Control: public, max-age=300
Vary: Accept-Language
```

Conceptually:

```
Cache key

/products?category=books
          +
Accept-Language=en
```

is different from:

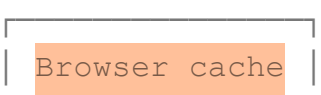
```
/products?category=books
          +
Accept-Language=fr
```

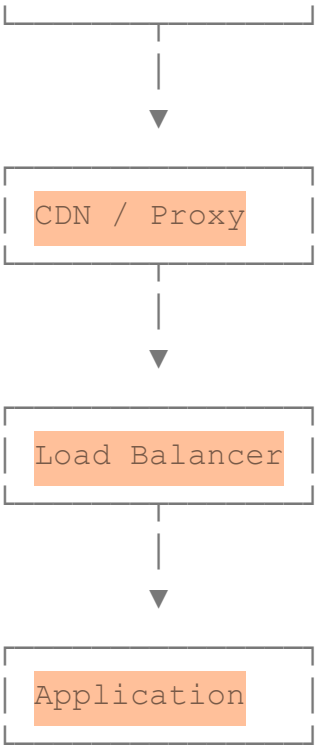
Poor cache-key design is one of the easiest ways to create correctness or security problems.

---

# 17. Browser caching vs CDN caching

Think about caching as layers:






`Cache-Control` lets you establish policies across these layers.

For example:

```
Cache-Control: public, max-age=30, s-maxage=300
```

means approximately:

```
Browser → 30 seconds
CDN     → 5 minutes
```

This is an extremely useful pattern for high-scale applications.

---

# 18. Common use-case recipes

Here is the practical cheat sheet I would use when designing an API/platform.

| Use case | Recommended starting point |
| --- | --- |
| Fingerprinted static assets | `public, max-age=31536000, immutable` |
| Public content, modest freshness | `public, max-age=300` |

| Use case | Recommended starting point |
| --- | --- |
| Public API, CDN preferred | `public, max-age=30, s-maxage=300` |
| User-specific data | `private, max-age=60` |
| User-specific data requiring validation | `private, no-cache` |
| Highly sensitive response | `no-store` |
| Cache but always validate | `no-cache` |
| Strong stale prohibition | `must-revalidate` |
| High availability public content | `stale-if-error=600` |
| Low-latency public content | `stale-while-revalidate=300` |
| Immutable versioned resource | `immutable` |

These are **starting points**, not universal rules. Your correctness requirements should determine the policy.

---

# 19. A useful API design matrix

For a REST API, I'd classify endpoints like this:

## A. Immutable/public

```
GET /api/countries
```

Potentially:

```
Cache-Control: public, max-age=86400
```

---

## B. Public and frequently changing

```
GET /api/news
```

Potentially:

```
Cache-Control: public, max-age=30, s-maxage=120, stale-while-revalidate=3
```

---

## C. User-specific

```
GET /api/me/orders
```

Potentially:

```
Cache-Control: private, max-age=30
```

If correctness is more important:

```
Cache-Control: private, no-cache
```

---

## D. Highly sensitive

```
POST /api/payment
```

Potentially:

```
Cache-Control: no-store
```

---

## E. Versioned static asset

```
/app.81a92d.js
```

```
Cache-Control: public, max-age=31536000, immutable
```

---

# 20. Don't confuse request and response directives

`Cache-Control` can appear on both requests and responses.

For example, a client can send:

```
Cache-Control: no-cache
```

meaning roughly:

> I want a fresh/revalidated response rather than blindly using a cached response.

Whereas the server might send:

```
Cache-Control: public, max-age=300
```

meaning:

> You may reuse this response for 300 seconds.

The semantics depend on whether the directive is being applied by the client or the response cache.

---

# 21. `max-age=0` is not the same as `no-store`

This is another subtle but important distinction.

```
Cache-Control: max-age=0
```

means the response immediately becomes stale.

It can still be stored and potentially revalidated.

Whereas:

```
Cache-Control: no-store
```

means don't store it.

And:

```
Cache-Control: no-cache
```

means it can be stored, but must be validated before reuse.

Think:

```
max-age=0
    ↓
stored, immediately stale

no-cache
    ↓
stored, validation required

no-store
    ↓
not stored
```

---

# 22. A good mental model

I recommend thinking about `Cache-Control` in four independent dimensions:

## 1. Can it be stored?

```
no-store
private
public
```

## 2. How long is it fresh?

```
max-age
s-maxage
```

### 3. What happens when it becomes stale?

```
must-revalidate
stale-while-revalidate
stale-if-error
```

### 4. Can it change during its lifetime?

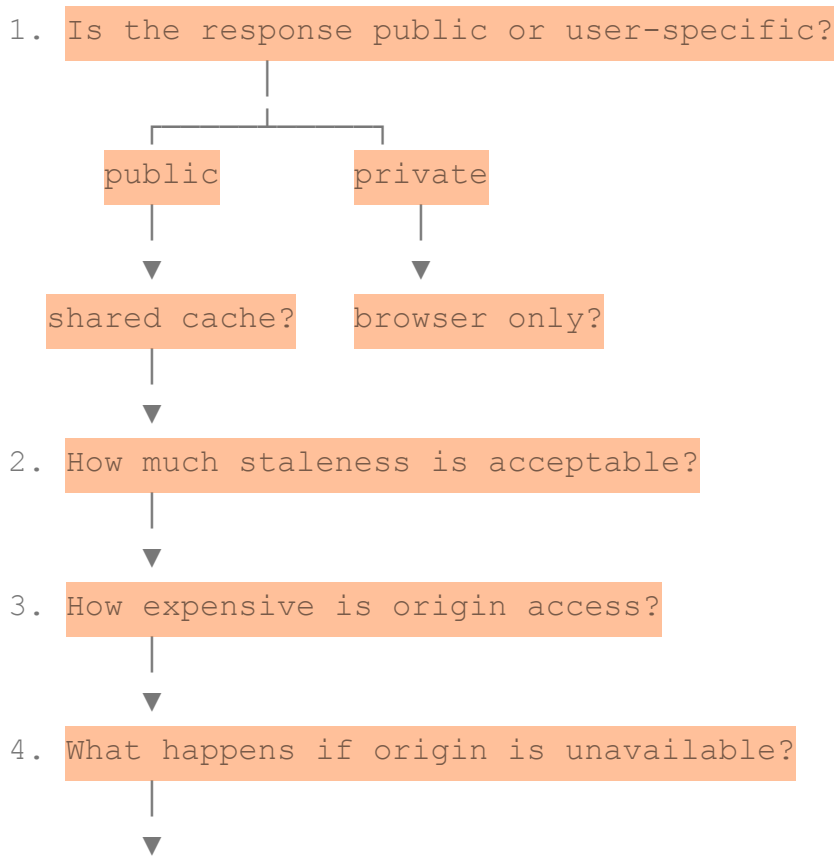
```
immutable
```

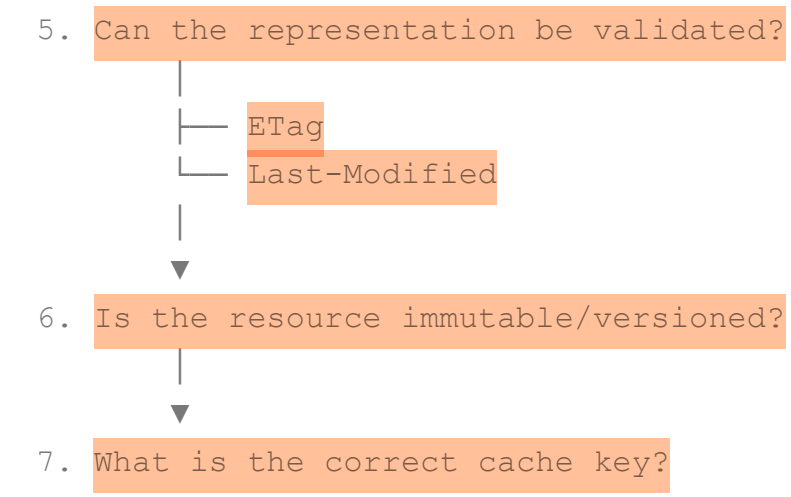
This makes complicated directives much easier to reason about.

---

# 23. The architectural decision process

When designing caching for an endpoint, I would ask:






That process is much more reliable than simply adding:

```
Cache-Control: max-age=3600
```

everywhere.

---

# 24. Recommended defaults for a modern system

For a modern web platform, I'd generally start with something like:

```
Immutable static assets
    public, max-age=31536000, immutable

Public APIs
    public, max-age=30, s-maxage=300

Public APIs where stale data is acceptable
    public, max-age=30,
    s-maxage=300,
    stale-while-revalidate=300,
    stale-if-error=600

User-specific APIs
    private, max-age=30

User-specific APIs requiring strict freshness
    private, no-cache
```

```
Sensitive responses
    no-store
```

Then tune based on:

- freshness requirements
- traffic volume
- origin cost
- CDN hit ratio
- latency SLOs
- failure behavior
- data sensitivity
- cache-key cardinality
- invalidation strategy

The key architectural principle is:

> **Caching is a correctness decision first and a performance optimization second.**

A fast cache that occasionally returns the wrong user's data is obviously unacceptable. A slightly slower cache with precisely defined freshness and isolation semantics is usually the better engineering choice.

If you're designing this for **MIAP or another distributed platform**, the next level is to look at `Cache-Control` together with **ETag,** `Vary` **, CDN caching, cache invalidation, surrogate keys, cache stampede prevention, and HTTP conditional requests**. Those pieces form a much more complete caching architecture than `Cache-Control` alone.